

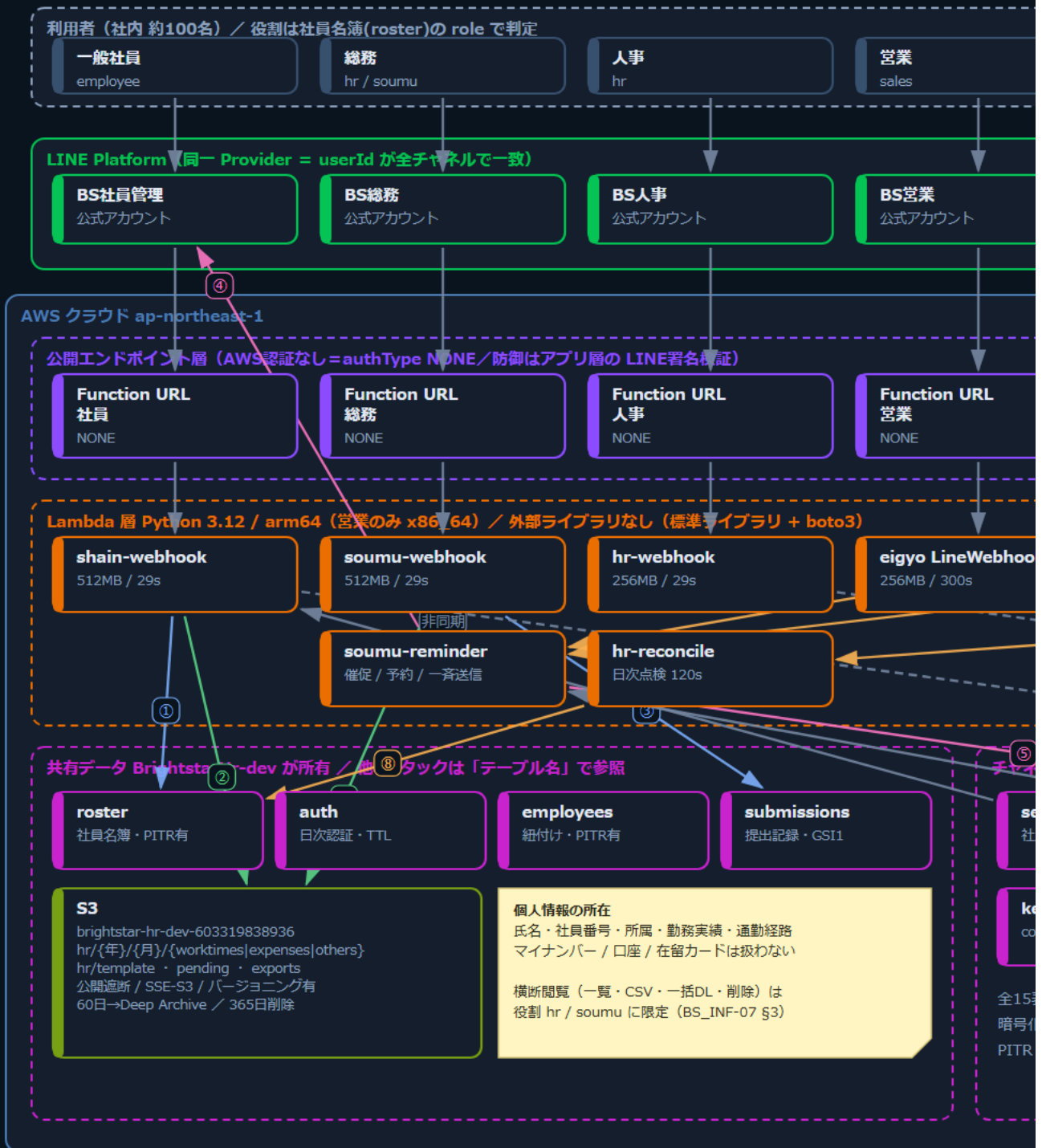
BrightStar 統合LINEアシスタント インフラ設計書（BS_INF-01～10）

本書は BS_INF シリーズ 10 分冊を 1 冊にまとめたものです。記載値はすべて 2026-09-19 時点の実デプロイを実機照会した結果です。

BrightStar 統合LINEアシスタント インフラアーキテクチャ

正 = BS_INF-01~10 設計書 / ap-northeast-1 ・ アカウント 603319838936 ・ stage=dev

実機照合日 2026-09-19 (Lambda 11本・DynamoDB 15表・S3 1本/デプロイ済みコードとリポジトリの md5 一致を確認済み)

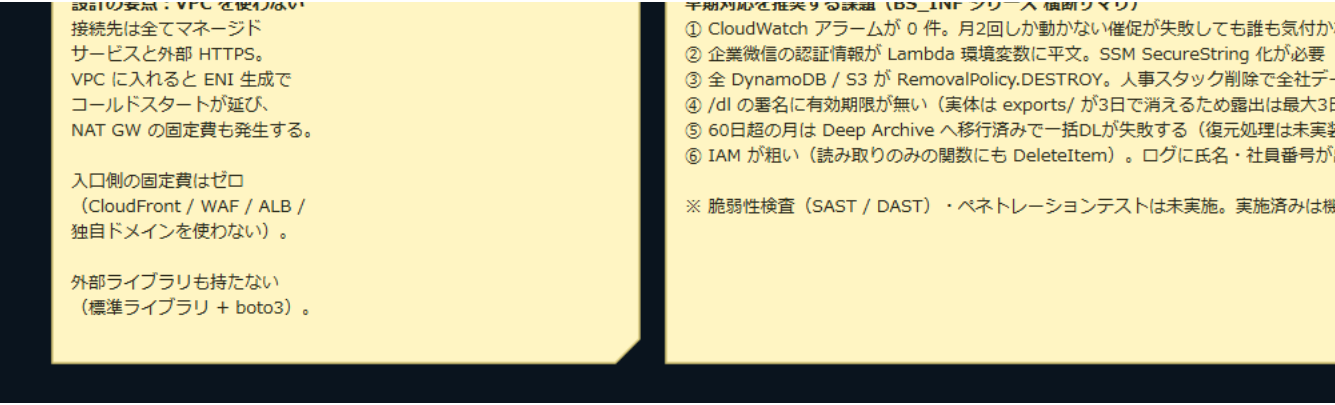


凡例 (線の色)

- ① ③ 同期リクエスト
- ② ⑥ S3 経路
- ④ ⑦ ⑧ 定期実行
- ⑤ push 配信
- 経路 (実線) / 非同期・参照 (破線)

処理フロー (線上の番号に対応)

- ① 本人確認: 「所属部署 お名前」→ roster 照合 → auth に当日認証 (TTL=当日)
- ② 提出: Excel / PDF / 画像 → S3 へ保存し submissions に記録
- ③ 回収: 総務が submissions の GSI1 (月×種別) を Query して提出状況を横断検索
- ④ 催促: EventBridge (25/28日) → reminder → 「社員チャネルの token」で postback
- ⑤ 既読確認: 社員が postback → broadcasts に確認記録 (配信IDは画面に出さない)
- ⑥ ダウンロード: ZIP/CSV を exports/ に生成 → 署名付き短縮URL → presigned URL
- ⑦ 予約催促: bookings を 10分間隔のポーラーが拾い、実行時点の未提出者のみへ
- ⑧ 日次点検: 毎日0:00 に到達性を確認し、到達不可なら紐付けを解除



BrightStar 統合LINEアシスタント インフラ設計書 (BS_INF シリーズ)

AWS サービス別に分冊したインフラ設計書。記載値はすべて 2026-09-19 時点の実デプロイを実機照会した結果であり、推測・設計当初の値ではない。記載を変更する場合は再照会のうえ更新すること。

対象	ap-northeast-1 / アカウント 603319838936 / ステージ dev
構成	5 チャンネル・5 スタック・Lambda 11本・DynamoDB 15表・S3 1本
検証	デプロイ済み Lambda とリポジトリの md5 一致を確認済み (BS_INF-02 §4)

設計書一覧

文書番号	文書名	主な内容
BS_INF-01	命名規則・タグ設計書	命名の基本形、タグ規則、既知の逸脱と対応方針
BS_INF-02	Lambda設計書	全11関数の諸元、外部ライブラリゼロ方針、vendoring、ソース同期検証
BS_INF-03	DynamoDB設計書	全15表、スタック跨ぎの共有設計、PITR、名簿のデータ設計
BS_INF-04	S3設計書	バケット設定、格納レイアウト、ライフサイクル、署名付き短縮URL
BS_INF-05	公開エンドポイント設計書	Function URL / API Gateway、LINE署名検証を軸にした3層防御
BS_INF-06	EventBridge設計書	定期実行4ルール、予約催促のポーラー方式
BS_INF-07	IAM設計書	ロール一覧、アプリ層の役割ベース認可、最小権限の評価
BS_INF-08	SSM・暗号化設計書	SecureString 方針、KMS 方針 (CMK 不採用)、鍵の用途分離
BS_INF-09	監視・ログ設計書	ログ保持期間、ログ中の個人情報、推奨アラーム構成
BS_INF-10	CDK構成管理・デプロイ設計書	スタック依存、必須context、安全な再デプロイ手順、落とし穴集

アーキテクチャ図: ../architecture/BS_インフラアーキテクチャ図.png (編集用: drawio)

応用層の文書 (別系統)

インフラではなく「何ができるか・どう動くか」は以下を参照。

文書	内容
10 システム概要書	5チャンネルの役割分担、全体アーキテクチャ
11 機能仕様書	チャンネル別の機能仕様
12 コードロジック説明書	リポジトリ構成、処理フロー、データモデル
01～05 利用マニュアル	講師・人事・営業・社員・総務 各向け

課題サマリ (全設計書の横断)

各設計書の「既知の課題」を深刻度順に集約したもの。

早期対応を推奨

深刻度	内容	出典
高	CloudWatch アラームが 0 件。催促失敗・webhook 障害を検知できない	BS_INF-09 § 4
高	企業微信の認証情報が Lambda 環境変数に平文	BS_INF-08 § 4
高	全 DynamoDB / S3 が RemovalPolicy.DESTROY。人事スタック削除で全社データ消失	BS_INF-03 § 7
中	ダウンロード用署名付きリンクに有効期限が無い（露出は最大3日）	BS_INF-04 § 6
中	60日超の月は Deep Archive により一括DLが失敗する	BS_INF-04 § 5

将来対応

深刻度	内容	出典
中	IAM の権限が粗い（読み取りのみの関数にも DeletetItem）	BS_INF-07 § 5
中	ログに氏名・社員番号が平文で出力される	BS_INF-09 § 3
中	CloudTrail データイベント未設定（提出物へのアクセス監査不可）	BS_INF-07 § 5
中	業務記録テーブルの PITR 未設定	BS_INF-03 § 7
中	CI/CD が無く手動デプロイ	BS_INF-10 § 7
中	エンドポイントにレート制限が無い	BS_INF-05 § 6
低	LINE シークレットを署名検証とURL署名で共用	BS_INF-08 § 2
低	運用者用 IAM ロール未整備（管理者ユーザーで直接運用）	BS_INF-07 § 5

実施済み（2026-09-19）

内容	出典
CloudWatch Logs の無期限保持を解消（全19ロググループに保持期間を設定）	BS_INF-09 § 2
旧スタック由来の孤立ロググループ5件を削除	同上
デプロイ済み Lambda とリポジトリの同期を機械検証（全11関数一致）	BS_INF-02 § 4

未実施のテスト

脆弱性検査（SAST / DAST）およびペネトレーションテストは実施していない。実施済みのテストは機能面の単体テスト・結合テスト（AWS をモックしたオフラインテスト）のみ。

検査を行う場合の候補： bandit （Python 静的解析）／npm audit （CDK 依存）／cdk-nag ・ checkov （IaC 検査）／AWS Security Hub ・ GuardDuty。

BrightStar 統合LINEアシスタント 命名規則・タグ設計書

項目	内容
文書番号	BS_INF-01
文書名	命名規則・タグ設計書
対象システム	BrightStar 統合LINEアシスタント（5チャンネル）
版数	初版
最終更新	2026-09-19
実機照合	2026-09-19（ap-northeast-1 / アカウント 603319838936）

1. 方針

リソース名は「アプリ名 + ステージ + 用途」の 3 要素で構成し、物理名を明示指定する。

理由：本システムは 5 つの独立スタックが同じ DynamoDB テーブル・S3 バケットを名前参照で共有する（BS_INF-03 §4）。CDK の自動生成名（EkiCommute-dev-StaffTable11B9C6C0-XMT2R0ORBPMC のようなハッシュ付き）では、他スタックから名前で参照できない。したがって共有対象は必ず物理名を固定する。

共有しないリソース（IAM ロール等）は CDK の自動生成名で構わない。名前を固定すると別環境へ並行デプロイできなくなるため、固定は必要なものに限る。

2. 環境・アカウント

項目	値
リージョン	ap-northeast-1（東京）－LINE 利用者・業務が日本国内のため
アカウント	603319838936（単一アカウント）
ステージ	dev のみ（本番相当として運用中）

現状の割り切り：環境分離（dev/stg/prod）は行っていない。ステージ名はリソース名に含めてあるため、別ステージの並行デプロイは `-c stage=stg` で可能な設計にはなっている。実運用は dev 1 本。

3. 命名規則

3.1 基本形

```
{app}-{stage}-{resource}
```

要素	値	例
{app}	brightstar-kenshu / brightstar-hr / brightstar-soumu / brightstar-shain / eki-commute	
{stage}	dev	
{resource}	用途を表す小文字英字	roster / webhook / submissions

例：brightstar-hr-dev-roster、brightstar-soumu-dev-webhook

3.2 共通ルール

- 小文字 + ハイフン区切り（kebab-case）。
- 略語は使わない（sub ではなく submissions）。
- 連番は付けない（1 リソース 1 用途のため）。

3.3 例外（AWS 側の制約・CDK の都合）

対象	実際の名前	理由
CloudFormation スタック（人事・総務・営業）	BrightstarHr-dev / BrightstarSoumu-dev / EkiCommute-dev	TypeScript CDK 側で PascalCase の Construct ID をそのままスタック名に使用したため
CloudFormation スタック（研修・社員）	brightstar-kenshu-dev / brightstar-shain-dev	Python CDK 側は基本形どおり
IAM ロール	BrightstarHr-dev-WebhookFunctionServiceRoleA0DBD641-H8SgG36XAmgI	CDK 自動生成。共有しないため固定不要
S3 バケット	brightstar-hr-dev-603319838936	バケット名はグローバル一意が必要なためアカウントIDを付加
営業の DynamoDB	EkiCommute-dev-StaffTable11B9C6C0-XMT2R0ORBPMC	CDK 自動生成名のまま（基本形からの逸脱・§5）

4. タグ規則

全リソースにスタック単位で以下を付与する（Tags.of(this).add(...)）。

タグキー	値	目的
Project	brightstar-kenshu / brightstar-hr / brightstar-soumu / brightstar-shain / eki-commute	Cost Explorer でのチャネル別コスト把握、削除時の一括特定

タグキー	値	目的
Stage	dev	環境識別
ManagedBy	cdk	手動作成リソースとの区別（手動作成物は削除時に取り残される）

実機のタグ付与状況（2026-09-19）

スタック	Project	Stage	ManagedBy	備考
brightstar-kenshu-dev	✔	✔	✔	規則どおり
BrightstarHr-dev	✔	✔	✔	規則どおり
BrightstarSoumu-dev	✔	✔	✔	規則どおり
brightstar-shain-dev	✔	✔	✔	規則どおり
EkiCommute-dev	✔	✖ Env=dev	✖ 無し	規則からの逸脱（\$5）

5. 既知の逸脱と対応方針

#	逸脱内容	影響	対応
1	営業スタックのタグが Env=dev（他は Stage=dev）、ManagedBy 無し	タグによる横断集計・一括抽出で営業だけ漏れる	将来対応。タグ変更は全リソース再作成を伴わないため、cdk deploy で追従可能
2	営業の DynamoDB テーブル名が CDK 自動生成	他スタックから名前参照できない。コンソールで用途が読み取れない	対応しない。営業のテーブルは他チャンネルと共有しない設計のため実害がなく、改名はテーブル再作成＝データ消失を伴う
3	スタック名の大文字小文字が不統一	表示上のみ。実害なし	対応しない。スタック名変更＝スタック作り直し＝全リソース再作成となり、リスクに見合わない

いずれも「規則を後から決めた」ことに起因する。新規リソースは基本形に従うこととし、既存の逸脱は上表のとおり影響度で判断する。

6. 関連文書

- リソースの実体一覧 … BS_INF-02 Lambda設計書 / BS_INF-03 DynamoDB設計書 / BS_INF-04 S3設計書
- スタック構成とデプロイ … BS_INF-10 CDK構成管理・デプロイ設計書

BrightStar 統合LINEアシスタント Lambda設計書

項目	内容
文書番号	BS_INF-02
文書名	Lambda設計書
対象システム	BrightStar 統合LINEアシスタント（5チャンネル）
版数	初版
最終更新	2026-09-19
実機照合	2026-09-19（全11関数）

1. 方針

項目	設計	理由
実行基盤	Lambda のみ（コンテナ・EC2 を使わない）	LINE Bot は「メッセージが来たときだけ動く」典型的なイベント駆動。常時起動リソースはコストの無駄で、パッチ運用も発生する
ランタイム	Python 3.12 全関数統一	チャンネル間で共通モジュール（認証・多言語・S3操作）をコピー共有しているため、ランタイムを揃えないと共有できない

項目	設計	理由
アーキテクチャ	arm64 (Graviton2) (営業のみ x86_64)	同性能で 約20%安い。営業だけ x86_64 なのは requests/BeautifulSoup を含む Layer を x86 でビルド済みのため (BS_INF-10 §5)
外部ライブラリ	使わない (標準ライブラリ + boto3 のみ)。営業のみ例外	①Layer ビルド・Docker が不要でデプロイが単純 ②依存パッケージ経由の脆弱性リスクが構造的に発生しない ③コールドスタートが速い
VPC	配置しない	接続先は全て AWS のマネージドサービス (DynamoDB/S3/SSM) と外部 HTTPS。VPC に入れると ENI 生成でコールドスタートが延び、NAT GW の固定費も発生する
同時実行数の予約	設定しない	社内 100 名規模・月次ピークでもアカウント既定 (1000) に遠く及ばない

xlsx を標準ライブラリだけで読む

提出された Excel の年月・氏名を読むために openpyxl を入れるとサイズも依存も増える。本システムは **xlsx = zip + XML** であることを利用し、`zipfile + xml.etree.ElementTree` で必要セルだけを読む自作モジュール (`common/xlsx.py`) を使う。これにより「外部ライブラリゼロ」の方針を崩さずに Excel を扱える。

2. 関数一覧 (2026-09-19 実測)

#	関数名	チャネル	アーキ	メモリ	タイムアウト	起動元
1	brightstar-kenshu-dev-LineWebhookFunction...	研修	arm64	1024MB	20s	API GW POST /line
2	brightstar-kenshu-dev-WebhookFunction...	研修	arm64	1024MB	20s	API GW GET/POST /wechat
3	brightstar-kenshu-dev-WebFunction...	研修	arm64	1024MB	20s	API GW POST /web/*
4	brightstar-kenshu-dev-ReminderFunction...	研修	arm64	1024MB	20s	EventBridge 10分間隔
5	brightstar-hr-dev-webhook	人事	arm64	256MB	29s	Function URL
6	brightstar-hr-dev-reconcile	人事	arm64	256MB	120s	EventBridge 毎日 0:00 JST
7	brightstar-soumu-dev-webhook	総務	arm64	512MB	29s	Function URL
8	brightstar-soumu-dev-reminder	総務	arm64	256MB	60s	EventBridge + 他 Lambda からの非同期 Invoke
9	brightstar-shain-dev-webhook	社員	arm64	512MB	29s	Function URL
10	EkiCommute-dev-LineWebhookFn...	営業	x86_64	256MB	300s	Function URL
11	EkiCommute-dev-ApiFn...	営業	x86_64	256MB	300s	Function URL (AWS_IAM)

※ 上記のほか、CDK が自動生成する補助関数 (`LogRetention*`・`CustomS3AutoDeleteObjects*`・`CustomCDKBucketDeployment*`) が存在する。設計対象外。

メモリ・タイムアウトの設計根拠

値	対象	理由
256MB	人事 webhook / 総務 reminder / 営業	テキスト応答と DynamoDB 読み書きのみ。メモリ増は速度に寄与しない
512MB	総務 webhook / 社員 webhook	一括DL の ZIP 生成をメモリ上で行う (全社員の提出物を読み込む) ため余裕を持たせる
1024MB	研修 全関数	Bedrock 呼び出し・RAG 検索を含み、メモリ増=CPU 増でコールドスタートと推論待ちを短縮
29s	全 webhook	LINE の応答タイムアウトが 30 秒。これを超えると LINE 側で失敗扱いになるため、直前で切って自前のエラー応答を返す
60s / 120s	総務 reminder / 人事 reconcile	全社員ループ+LINE API 逐次呼び出し。人数に比例するため長め
300s	営業	外部サイト (駅探) のスクレイピングを含み、応答が遅い場合がある

重い処理は同期で返さない：催促送信・一斉送信・経路取得は、webhook が即座に ack を返し、実処理は別 Lambda を非同期 Invoke (`InvocationType=Event`) して push で結果を届ける。29 秒の壁を業務側に露出させないための共通パターン。

3. コード構成と vendoring

3.1 共通モジュールの配置

common/ 配下（authlib.py 認証 / assist.py 多言語・QuickReply / line.py LINE API / business.py 業務 / s3util.py / i18n.py / xlsx.py）は、各チャンネルのディレクトリに実体をコピーして配置している。

項目	設計	理由
共有方式	コピー（vendoring）。Lambda Layer や共有パッケージにしない	①Layer にすると 5 スタックが 1 つの Layer バージョンに結合し、1 チャンネルの修正が全チャンネルのデプロイを要求する ②チャンネル独立（1 チャンネルの障害・改修が他へ波及しない）を最優先した
代償	同じ修正を複数箇所に適用する必要がある	修正漏れを防ぐため、 正本を jinji/lambda/common/ とし、他チャンネルへ反映する運用とする

⚠ **実装上の注意**：社員の提出処理が実際に動くのは shain/lambda/jinji/common/ にコピーされた方である。jinji/lambda/common/ だけ直しても社員チャンネルの挙動は変わらない。

3.2 社員チャンネルの名前空間分離

社員チャンネル（shain）は研修と総務の「社員側」機能を 1 つの Lambda に同梱する。両者に同名の common/ handlers/ が存在するため、kenshu.* / jinji.* の名前空間に分けて import することで衝突を回避している。

```
shain/lambda/  
├── handler.py          ルーター（モードで振り分け）  
├── kenshu/{common, handlers}/  
└── jinji/{common, handlers}/
```

I/O 層（LINE トークン・SSM パラメータ）は社員チャンネルのもの 1 本に寄せており、vendoring 元のチャンネルのトークンは使わない。

4. ソースとデプロイの同期

CDK の Code.fromAsset がリポジトリのディレクトリをそのまま zip 化して配布する。ビルド工程が挟まらないため、リポジトリの内容 = デプロイされたコードが常に成立する。

チャンネル	fromAsset が指す先
研修	kenshu/src
人事	jinji/lambda
総務	soumu/lambda
社員	shain/lambda
営業	eigyo/infra/build/lambda （build.py が各所から集約。このディレクトリは.gitignore）

検証手順（納品時に実施）

各関数の Code.Location（S3 署名付き URL）から zip を取得・展開し、リポジトリ側とファイル単位の md5 比較を行う。営業のみ build.py のコピー規則で zip 内パス → git 管理下の元ファイルへ読み替えて突合する。

2026-09-19 実施結果：全11関数・計122ファイルが一致（差分なし）

チャンネル	関数数	比較ファイル数	結果
研修	4	24	一致
人事	2	15	一致
総務	2	15	一致
社員	1	43	一致
営業	2	25	一致

5. 落とし穴

#	内容	対策
1	研修スタックの再デプロイで企業微信の設定が消える。WeCom 系の値は Lambda 環境変数にしか無く、-c で渡さないと空文字で上書きされる	デプロイ前に稼働中の Lambda 環境変数から値を読み戻して -c を組み立てる。デプロイ後に before/after を突合して欠落が無いことを確認する（BS_INF-10 §4）
2	人事・総務スタックは -c masterHrPrefix 等を渡さないと環境変数が空になる	同上。必須 context は BS_INF-10 §3 の表を参照

#	内容	対策
3	vendoring 先を直し忘れる（ jinji/lambda/common だけ直して社員の挙動が変わらない）	修正時は shain/lambda/jinji/common/ の同名ファイルも必ず確認する

6. 費用

Lambda はリクエスト課金 + 実行時間課金で、社内 100 名規模では無料枠（月100万リクエスト・40万GB秒）に収まる見込み。arm64 採用により単価はさらに 20% 低い。コストの主体は Lambda ではなく CloudWatch Logs と DynamoDB（BS_INF-09 § 5 / BS_INF-03 § 6）。

7. 関連文書

- 公開方法（Function URL / API Gateway）… BS_INF-05
- 権限 … BS_INF-07 IAM設計書
- 定期実行 … BS_INF-06 EventBridge設計書
- デプロイ … BS_INF-10

BrightStar 統合LINEアシスタント DynamoDB設計書

項目	内容
文書番号	BS_INF-03
文書名	DynamoDB設計書
対象システム	BrightStar 統合LINEアシスタント（5チャンネル）
版数	初版
最終更新	2026-09-19
実機照合	2026-09-19（全15テーブル）

1. 方針

項目	設計	理由
DB 種別	DynamoDB のみ （RDS を使わない）	アクセスパターンが「社員番号で引く」「ユーザーIDで引く」の単純なキー参照に限られ、結合・集計を必要としない。RDS は VPC・固定費・バックアップ運用を伴い、この規模では過剰
課金	全テーブル オンデマンド（PAY_PER_REQUEST）	トラフィックが月末に極端に偏る（25日・28日の催促、月初の提出集中）。プロビジョンド方式ではピークに合わせた設定が平常時の無駄になる
暗号化	AWS 管理キー（KMS） 。CMK は作らない	CMK の価値＝キーポリシーによるアクセス分離・独自ローテーション。現時点で該当要件がなく、キー毎の固定費と鍵運用事故（無効化＝復号不能）のリスクを負う理由がない
削除ポリシー	RemovalPolicy.DESTROY	検証段階の設計。 本番運用として継続するなら RETAIN へ変更すべき （§ 7 課題2）
テーブル共有	社員名簿・認証は スタックを跨いで名前参照で共有	5 チャンネルで「同じ人」を一元管理するため（§ 4）

2. テーブル一覧（2026-09-19 実測）

すべて PAY_PER_REQUEST / 暗号化有効。

テーブル	所有スタック	PK（+SK）	GSI	TTL	PITR	用途
brightstar-hr-dev-roster	人事	empId	—	—	✓	社員名簿。全チャンネルの認証の基点
brightstar-hr-dev-auth	人事	pk（channel#userId）	—	✓	—	日次認証・言語設定（当日限り）
brightstar-hr-dev-employees	人事	userId	—	—	✓	LINEユーザー↔社員の紐付け状態
brightstar-hr-dev-submissions	人事	userId + sk	GSI1	—	✓	提出記録

テーブル	所有スタック	PK (+SK)	GSI	TTL	PITR	用途
brightstar-soumu-dev-bookings	総務	bookingId	—	—	—	催促予約
brightstar-soumu-dev-broadcasts	総務	bcastId	—	—	—	配信バッチ・既読確認状況
brightstar-shain-dev-session	社員	userId	—	—	—	当日のモード（研修／総務）
brightstar-kenshu-dev-courses	研修	courseId	—	—	—	研修コース
brightstar-kenshu-dev-enrollments	研修	openid + courseId	GSI1	—	—	受講申込
brightstar-kenshu-dev-groups	研修	courseId + groupId	—	—	—	分組
brightstar-kenshu-dev-students	研修	openid	—	—	—	受講者
brightstar-kenshu-dev-results	研修	openid + itemKey	GSI1	—	—	受講結果
brightstar-kenshu-dev-knowledge	研修	docId + chunkId	—	—	—	RAG 用ナレッジ
EkiCommute-dev-StaffTable…	営業	staff_id	—	—	—	要員名簿
EkiCommute-dev-CacheTable…	営業	cacheKey	—	✓	—	経路検索キャッシュ

営業の 2 テーブルのみ **AWS 所有キー**（他は AWS 管理 KMS キー）。AWS 所有キーは CloudTrail に鍵利用が記録されず、キーの存在をコンソールで確認できない。実害は小さいが**監査性は劣る**。

3. 主要テーブルのデータ設計

3.1 roster（社員名簿） — 本システムの中核

全チャネルの「この人は誰で、何をしてよいか」の唯一の判断材料。

属性	型	説明
empId (PK)	S	社員番号。実データは会社の 10 桁社員番号
name	S	氏名（漢字）
department	S	所属部署
attribute	S	部署の細分類（例：日本籍／外国籍）
role	S	主役割： employee / hr / soumu / teacher / sales
roles	S	複数役割（CSV）。1 人に複数付与する場合に使用
admin	BOOL	一斉送信の実行権限
aliases	S	別名（カタカナ・ローマ字・簡体/繁体字）。氏名照合の表記ゆれ吸収用
lineUserId	S	紐付いた LINE ユーザー ID（1社員番号=1アカウントの占有ロック）
blocked	BOOL	bot のブロック／削除を検知したフラグ
manualOk	M	{period}#{type} → {by, at, note}。メール等で受領済みの手動「済」記録（監査情報つき）

認可の設計：チャネルのゲートは role / roles を見て判定する（authlib.has_role()）。役割を持たない社員が総務チャネルを操作することはできない。個人情報（提出物）へのアクセスは**総務・人事の役割を持つ者に限定**される。

3.2 submissions（提出記録）

属性	説明
userId (PK)	LINE ユーザー ID
sk (SK)	{period}#{type}。その他経費のみ {period}#other#{ts}（月内複数件を許すため）
gsi1pk / gsi1sk	{period}#{type} / userId。 GSI1 = 月×種別で全社員を横断するためのインデックス
s3Key / fileName / submittedAt	実体の所在と受付情報

GSI1 の設計理由：総務の「未提出確認」は「2026-07 の勤務表を出した人の一覧」を必要とする。メインテーブルは userId が PK なので全件 Scan になってしまう。GSI1 で period#type を PK にすることで **Query 1 回**で取得できる。

3.3 auth（日次認証）

pk = {channel}#{userId} とし、チャンネルごとに独立した認証行を持つ。expireAt (TTL) により当日限りで自動失効する。

TTL を使う理由：認証を永続化すると「退職者の LINE が生きたまま」になる。当日限りにすることで、翌日には必ず名簿照合が再実行され、名簿から消えた人は自動的に使えなくなる。

4. スタックを跨いだテーブル共有



項目	設計	理由
参照方式	テーブル名の文字列で参照 (Table.fromTableName)。CloudFormation の Export/Import を使わない	Export/Import はスタック間に削除順序の依存を作る。人事スタックを更新するたびに参照側がロックされるのを避けたい
前提	テーブル名が固定であること	BS_INF-01 §1 で物理名を固定している理由がこれ
リスク	人事スタックを destroy すると他 4 チャンネルが全滅する	運用上の注意事項として明記。RemovalPolicy の見直しとあわせて対応 (§7 課題2)

同一 LINE Provider 前提：5 チャンネルは同じ LINE Provider 配下にあるため、同一人物の userId が全チャンネルで一致する。これにより 1 チャンネルで本人確認すれば他チャンネルでも本人と判定できる。

5. バックアップ

テーブル	PITR	判断
roster / employees / submissions	有効	名簿の誤削除・提出記録の誤操作は業務影響が大きく、復旧手段が必要
auth / session	無効	当日限りの一時データ。失っても翌日の再認証で復旧する
bookings / broadcasts	無効	要検討 (§7 課題1)。予約催促・既読確認は業務記録としての性質を持つ
研修系 6 テーブル	無効	要検討 (§7 課題1)。コース・受講申込は業務記録
営業 2 テーブル	無効	StaffTable は要員名簿＝業務記録。CacheTable は再取得可能なので不要

6. 費用

オンデマンド課金。実データ量は 2026-09 時点で全テーブル合計 100 件未満 (roster 18・submissions 25 など) で、読み書き回数も社内 100 名規模では無料枠内。PITR を有効にしたテーブルはストレージ量に対する追加課金が発生するが、データ量が極小のため無視できる。

7. 既知の課題

#	深刻度	区分	内容	推奨対応	要否
1	中	データ保護	bookings / broadcasts / 研修6表 / 営業 StaffTable の PITR 未設定。誤削除・不正な一括更新から復旧できない	業務記録の性質を持つテーブルに PITR を有効化 (pointInTimeRecovery: true)。コストはデータ量比例で極小	将来対応
2	高	構成	全テーブルが RemovalPolicy.DESTROY。スタック削除でデータが消える。特に人事スタックは 4 チャンネルの共有テーブルを保持しており、誤削除の影響が全社に及ぶ	本番相当で運用を継続するなら RETAIN へ変更する。変更は CFN のメタデータのみでデータ影響なし	早期対応推奨
3	低	監査	営業 2 テーブルのみ AWS 所有キーで、鍵利用が CloudTrail に残らない	他と揃えて AWS 管理キーへ。テーブル再作成が必要なため、実害の小ささから対応しない判断も可	将来対応

8. 関連文書

- 暗号化方針の全体 … BS_INF-08 SSM・暗号化設計書

- テーブルへのアクセス権限 … BS_INF-07 IAM設計書
- 命名規則 … BS_INF-01

BrightStar 統合LINEアシスタント S3設計書

項目	内容
文書番号	BS_INF-04
文書名	S3設計書
対象システム	BrightStar 統合LINEアシスタント（5チャンネル）
版数	初版
最終更新	2026-09-19
実機照合	2026-09-19

1. 方針

バケットは**1本のみ**（brightstar-hr-dev-603319838936）とし、プレフィックスで用途を分ける。

理由：バケットを分けると、ライフサイクル・暗号化・パブリックアクセスブロックの設定を**複数箇所で維持**することになり、設定漏れの事故が起きやすい。本システムは保管対象が「社員が提出したファイル」の一種類に集約されるため、1バケット＋プレフィックス分割で十分。

社員の提出物は**氏名・勤務実績・通勤経路・住所を含む個人情報**である。したがってバケットは**完全非公開を前提**とし、配布は署名付きの一時 URL でのみ行う（§4）。

2. バケット設定（2026-09-19 実測）

項目	値	理由
バケット名	brightstar-hr-dev-603319838936	名前がグローバル一意である必要があるためアカウントIDを付加
所有スタック	BrightstarHr-dev（人事）	提出物の基盤を人事スタックに集約。社員が書き、総務が読む
パブリックアクセスブロック	4項目すべて有効	個人情報を含むため公開経路を構造的に塞ぐ
既定暗号化	SSE-S3（AES-256）	KMS CMK は使わない（BS_INF-08 §2）
バージョンング	有効	提出物の上書き（再提出）と誤削除からの復旧手段。旧版が残る
転送中の暗号化	HTTPS のみ（Lambda・署名付き URL とも）	

3. オブジェクトレイアウト

hr/{yyyy}/{mm}/worktimes/	勤務表（{社員番号}_{氏名}）_{YYYYMM}.xlsx
hr/{yyyy}/{mm}/expenses/	交通費経費【{YYYY}年{MM}月{社員番号}_{氏名}】.xlsx
hr/{yyyy}/{mm}/others/	{用途}_{氏名}_{period}_{ts}.{ext}
hr/template/	配布用テンプレート（勤務表.xlsx / 交通費経費.xlsx）
pending/	チェック不合格ファイルの一時退避
exports/	一括DL の ZIP・CSV 出力

3.1 ファイル名の設計

項目	設計	理由
アップロード時の元ファイル名	使わない 。システムが付け直す	社員が任意の名前で送ってくるため、そのままでは整理できない。また同名衝突が起きる
社員番号をファイル名に含める	含める	同姓同名の取り違えを防ぐ 。氏名だけでは一意にならない
格納する年月	ファイルの中身から判定 （勤務表＝J1「年」/ L1「月」、交通費＝B1）。読めない場合は提出月	月末締め書類が翌月頭にずれ込んでも、 書類に書かれた月のフォルダ に入る

3.2 上書きの扱い

ケース	上書き	理由
同一社員・別の月	されない	年月がパスとファイル名の両方に入るため、キーが別になる
同一社員・同月・同種別	される	「再提出＝最新が有効」という業務ルールの実装。バージョンングにより旧版は S3 に残る
その他経費	されない	ファイル名にタイムスタンプを含め、月内複数件の提出を許す

4. 配布経路（署名付き短縮 URL）

LINE のボタンに設定できる URL は **1000 文字**までだが、S3 の署名付き URL はこれを超えることがある。そのため**2段構え**にしている。

LINE のボタン

→ Lambda Function URL GET /dl?key=<S3キー>&sig=<HMAC>

① sig を HMAC-SHA256（鍵=LINE チャンネルシークレット）で検証

② S3 presigned URL（有効期限 1 時間）を発行して 302 リダイレクト

→ S3 から直接ダウンロード

項目	設計	理由
署名アルゴリズム	HMAC-SHA256、先頭16桁（64bit）	鍵は SSM SecureString の LINE チャンネルシークレットを流用
署名対象	S3 キーのみ	—
S3 presigned URL の有効期限	3600 秒	

⚠ 設計上の弱点（\$ 6 課題1）

署名対象に**有効期限が含まれていない**ため、`/dl` のリンク自体は**失効しない**。URL を知る者は誰でもダウンロードできる。

ただし `/dl` が指す先は**すべて `exports/` 配下**（ZIP・CSV）であり、ライフサイクルで**3 日後に自動削除**される。したがって実際の露出期間は最大 3 日に限定される。それでも、LINE メッセージの転送・端末の閲覧履歴・社内プロキシのログ経由で URL が漏れれば、その 3 日間は認証なしで**全社員の提出物 ZIP** が取得可能である。

5. ライフサイクル

ルール	対象	設定	理由
expire-pending	<code>pending/</code>	7日で削除	チェック不合格ファイルの一時退避。再提出されれば不要
expire-exports	<code>exports/</code>	3日で削除	配布用の一時生成物。 署名付きリンクの実質的な有効期限を兼ねる
submissions-archive-expire	タグ <code>lifecycle=managed</code> の全オブジェクト	60日で Deep Archive へ移行 / 365日で削除	提出物は提出直後～翌月までしか参照されない。長期保管はコストを最小化しつつ、法定保存期間を意識して 1 年で廃棄

⚠ Deep Archive 移行による運用影響（\$ 6 課題2）

Deep Archive のオブジェクトは `GetObject` が**直接できない**（`InvalidObjectState`）。復元（Restore）操作を行い、数時間～半日待つ必要がある。

本システムの一括DLは対象月の全オブジェクトを `get_object` で読んで ZIP 化するため、**60日を超えた月を一括DLするとエラーになる**。コードに復元処理は実装していない。

例：7月分の提出物は 2026-10 頃に Deep Archive へ移行し、以後 `一括DL 202607` が失敗する。

6. 既知の課題

#	深 刻 度	区分	内容	推奨対応	要否
1	中	機微 情報	<code>/dl</code> の署名に有効期限が無く、URL を知る者が認証なしでダウンロードできる。対象は <code>exports/</code> の ZIP・CSV= 全社員の提出物や氏名一覧	署名対象に <code>exp</code> （失効時刻）を含め、 <code>/bcast</code> と同じ方式（ <code>hmac(uid:exp)</code> ）に揃える。実装は数行。露出は現状 3 日に限定されているため即時ブロックカーではない	早期 対応 推奨
2	中	運用	60日超の月の一括DL が Deep Archive により失敗する	①保存期間の方針を見直して移行日数を延ばす、② <code>restore_object</code> と復元待ちの案内を実装する、のいずれか。 実際に7月分が失敗し始める前	早期 判断

#	深刻度	区分	内容	推奨対応	要否
				に判断が必要	
3	低	構成	バケットが RemovalPolicy.DESTROY + autoDeleteObjects。スタック削除で提出物が全消去される	本番運用を継続するなら RETAIN へ	将来対応

7. 費用

保管量は 2026-09 時点で **数MB 程度**（提出物 9 件・テンプレ 5 件）。60 日で Deep Archive（保管単価が約 1/20）へ移行するため、社員数が増えても保管費は小さい。費用面ではなく**個人情報の保持期間**の観点でライフサイクルを設計している。

8. 関連文書

- 暗号化方針 … BS_INF-08
- /dl を提供するエンドポイント … BS_INF-05
- S3 へのアクセス権限 … BS_INF-07 IAM設計書

BrightStar 統合LINEアシスタント 公開エンドポイント設計書（Function URL / API Gateway）

項目	内容
文書番号	BS_INF-05
文書名	公開エンドポイント設計書
対象システム	BrightStar 統合LINEアシスタント（5チャンネル）
版数	初版
最終更新	2026-09-19
実機照合	2026-09-19

1. 方針

項目	設計	理由
公開方式	原則 Lambda Function URL 。研修チャンネルのみ API Gateway (HTTP API)	Function URL は追加費用ゼロ・構成要素ゼロで HTTPS 入口が手に入る。API GW が必要になるのは 1つのドメインで複数の入口を出し分けたい場合のみ
認証方式	authType=NONE（AWS 認証なし） + アプリ層で LINE 署名検証	LINE Platform は Webhook を送る際に SigV4 を付けられない 。AWS_IAM を設定すると LINE からのリクエストが全て 403 になる。したがって AWS 層では通し、 アプリ層で HMAC-SHA256 署名を必ず検証 する
CloudFront / WAF	使わない	入口は LINE Platform からの Webhook のみで、一般利用者がブラウザで叩く画面が無い。WAF の主目的（Web アプリへの攻撃遮断）に該当しない。また AWS WAF は HTTP API (v2) にアタッチできない
カスタムドメイン	設定しない	URL を目にするのは LINE Developers の設定画面のみ。独自ドメイン・ACM 証明書の運用コストに見合わない
CORS	設定しない	ブラウザからのクロスオリジン呼び出しが発生しない。「不要なのに開ける」事故を避ける

2. エンドポイント一覧（2026-09-19 実測）

2.1 Lambda Function URL

チャンネル	URL	認証	呼び出しモード
人事	https://z5kzr2rgvkd4vxgkyepd3z5crq0spbsw.lambda-url.ap-northeast-1.on.aws/	NONE	BUFFERED
総務	https://zguhjjngs3hn6uwrs37mbsvtju0nkqcj.lambda-url.ap-northeast-1.on.aws/	NONE	BUFFERED

チャンネル	URL	認証	呼び出しモード
社員	https://juxjey4vihgddhuurpbctb6ha0uunj1.lambdurl.ap-northeast-1.on.aws/	NONE	BUFFERED
営業 (LINE)	https://iz5o26r4nifhjbtsr2n5b37gy0nvcc.lambdurl.ap-northeast-1.on.aws/	NONE	BUFFERED
営業 (API)	https://cu2okzssncdc46kzqedf6xn3de0gdcfy.lambdurl.ap-northeast-1.on.aws/	AWS_IAM	BUFFERED

CORS はいずれも未設定。

営業の API 用 Function URL だけ **AWS_IAM** なのは、LINE ではなく**機械クライアント（社内ツール）**から呼ぶ想定であり、SigV4 を付けられるため。付けられる経路には必ず **AWS 認証**を掛ける、という原則を適用している。

2.2 API Gateway（研修チャンネルのみ）

API ID 2xqgja49x8 / HTTP API / ステージ \$default （autoDeploy 有効）

ルート	認証	統合先	用途
POST /line	NONE	LineWebhookFunction	LINE Webhook
GET /wechat	NONE	WebhookFunction	企業微信 URL 検証（echostr 応答）
POST /wechat	NONE	WebhookFunction	企業微信 メッセージ受信
POST /web/login	NONE	WebFunction	受講者向け Web ログイン
POST /web/submit	NONE	WebFunction	受講者の回答送信
POST /web/results	NONE	WebFunction	結果取得
POST /web/my-results	NONE	WebFunction	自分の結果取得

研修だけ **API Gateway** を使う理由：LINE・企業微信・Web 画面という**3 系統の入口を 1 つのドメインにまとめる**必要があった。Function URL は 1 関数 1 URL のため、入口ごとにドメインが変わってしまい、企業微信側のコールバック設定や Web 画面の呼び先が煩雑になる。

3. 認証・アクセス制御の層構造

エンドポイント自体は公開だが、**3 層で守る**。

層	方式	内容
① 送信元検証	LINE 署名検証（HMAC-SHA256）	リクエストヘッダ X-Line-Signature と、チャンネルシークレットで計算した署名を hmac.compare_digest で比較。不一致は即 401。 LINE 以外からのリクエストはここで全て落ちる
② 本人確認	社員名簿（roster）照合	「所属部署 お名前」で名簿と突合し、在籍者本人としてのみ登録を許す。 1 社員番号=1 LINE アカウントの占有ロック で成りすまし・二重登録を防ぐ。認証は TTL により 当日限り
③ 認可	役割（role / roles）によるチャンネルゲート	総務チャンネルは hr または soumu、研修（講師）は teacher、営業は sales を要求。役割を持たない社員は操作できず、他チャンネルへ誘導される。一斉送信はさらに admin を要求

署名検証が単一の防波堤である点は認識しておく必要がある。チャンネルシークレットが漏れれば、任意の偽装リクエストを送れる。シークレットは SSM SecureString にのみ保持し、ソース・環境変数に置かない（BS_INF-08）。

4. 副次エンドポイント（同一 Function URL 上の GET 経路）

webhook 用の Function URL は、POST（LINE Webhook）以外に以下の GET/POST 経路も処理する。

経路	チャンネル	認証	用途
GET /dl?key=…&sig=…	人事・総務・社員	HMAC 署名 （AWS 認証なし）	ZIP・CSV・提出物のダウンロード。S3 presigned URL へ 302（BS_INF-04 § 4）
GET /bcast?uid=…&exp=…&sig=…	総務	HMAC 署名（有効期限30分つき）	一斉送信の入力フォーム（HTML）。PC から長文を書くため
POST /bcast	総務	同上	フォーム送信

/bcast は署名に**失効時刻を含めている**が、/dl は含めていない。同じ仕組みなので /dl も揃えるべき（BS_INF-04 § 6 課題1）。

5. 制約と回避設計

制約	値	回避設計
LINE の応答タイムアウト	30 秒	全 webhook のタイムアウトを 29 秒 に設定。重い処理（催促送信・一斉送信・経路取得）は ack を即返し、別 Lambda を非同期 Invoke して結果を push で届ける
LINE ボタンの URL 長	1000 文字	署名付き短縮 URL（ /dl ）を挟んで presigned URL ヘリダイレクト
LINE reply token	1 回・数十秒のみ有効	複数回送る場合は reply ではなく push を使う。QuickReply 付きの push は <code>push_message</code> を使用
Function URL のレート制限	アカウント同時実行数に依存	社内 100 名規模では問題にならない。 予約同時実行数は設定していない

6. 既知の課題

#	深刻度	区分	内容	推奨対応	要否
1	中	構成	Function URL・API Gateway とも レート制限が無い 。署名検証前の大量リクエストで Lambda が無制限にスケールし、費用が跳ねる（署名不一致でも Lambda は起動している）	API GW 側はステージのスロットリング設定、Function URL 側は予約同時実行数の設定で上限を作る	将来対応
2	低	監査	API Gateway の詳細メトリクス（ DetailedMetricsEnabled ）が無効。ルート別の呼び出し状況が見えない	有効化（追加費用は軽微）	将来対応
3	低	構成	Function URL は 削除・再作成で URL が変わる 。変わると LINE Developers 側の Webhook URL 設定を手で直す必要がある	運用手順として明記済み（BS_INF-10 § 6）	対応不要（周知のみ）

7. 費用

Function URL は**追加費用なし**（Lambda の課金のみ）。API Gateway (HTTP API) はリクエスト課金（100万件あたり約 \$1.0）で、社内利用の規模では無視できる。CloudFront・WAF・ALB・独自ドメインを使わないため、**入口側の固定費はゼロ**。

8. 関連文書

- Lambda 本体 … BS_INF-02
- ダウンロード経路 … BS_INF-04 S3設計書
- 認証情報の保管 … BS_INF-08

BrightStar 統合LINEアシスタント EventBridge設計書

項目	内容
文書番号	BS_INF-06
文書名	EventBridge設計書
対象システム	BrightStar 統合LINEアシスタント（5チャンネル）
版数	初版
最終更新	2026-09-19
実機照合	2026-09-19

1. 方針

項目	設計	理由
スケジューラ	EventBridge ルール（cron / rate）	Lambda を時刻起動する最も単純な方法。追加のインフラを持たない

項目	設計	理由
EventBridge Scheduler	使わない	1回限りの予約実行には Scheduler が適するが、 予約1件ごとに AWS リソースが増える 運用を避けたい。本システムは予約を DynamoDB に持ち、 固定間隔のポーラー が拾う方式にした（§3）
イベントバス	既定バス（default）のみ	カスタムバス・イベント連携は不要
タイムゾーン	cron は UTC 指定 。JST 換算を式に織り込む	EventBridge の cron は UTC 固定。Lambda 側は環境変数 TZ=Asia/Tokyo で JST として動作する

2. ルール一覧（2026-09-19 実測）

ルール名	スケジュール（UTC）	JST	起動先	用途
brightstar-soumu-dev-reminder-schedule	cron(0 0 25,28 * ? *)	毎月 25日・28日 9:00	brightstar-soumu-dev-reminder	未提出者への自動催促
brightstar-soumu-dev-booking-poller	rate(10 minutes)	10分ごと	brightstar-soumu-dev-reminder	催促予約の期限到来分を実行
brightstar-hr-dev-reconcile-schedule	cron(0 15 * * ? *)	毎日 0:00	brightstar-hr-dev-reconcile	LINE 紐付けの到達性点検
brightstar-kenshu-dev-ReminderTick...	rate(10 minutes)	10分ごと	brightstar-kenshu-dev-ReminderFunction	研修の開講1時間前リマインド

すべて **ENABLED**。

スケジュール値の根拠

ルール	根拠
25日・28日 9:00	月末締め書類に対し、 締切前に 2 回 の猶予を作る。9:00 は始業直後で気付きやすい。UTC 0 0 が JST 9:00 に相当
毎日 0:00（UTC 15:00）	日付が変わるタイミング＝認証テーブルの TTL 失効と同期。退職・ブロックされたアカウントを翌営業日までに掃除する
10分間隔	予約催促・研修リマインドの 許容遅延 。「9:00 に送る」予約が最大 9 分遅れて 9:09 になっても業務上問題ない。1分間隔にすると呼び出し回数が 10 倍になり、得るものが無い

3. 予約催促の実行方式（ポーラー方式）

総務が「催促予約 2026-07-25 09:00」
→ bookings テーブルに { bookingId, runAtEpoch, status: pending } を登録
↓
10分ごとの booking-poller → reminder Lambda を trigger:"poll" で起動
→ runAtEpoch <= now かつ status=pending を抽出して実行
→ 実行時点の未提出者のみに送信 → status=sent

項目	設計	理由
予約の保持先	DynamoDB bookings	予約ごとに AWS リソース（Scheduler・ルール）を作らない。 予約の一覧・取消が DynamoDB の操作だけで完結 する
実行判定	ポーラーが runAtEpoch <= now で抽出	
送信対象	実行時点 で未提出の人だけ	予約時点の未提出者に送ると、その後に提出した人にも催促が飛んでしまう

4. Lambda 起動の経路

reminder Lambda は**3 つの経路**から起動される。イベントの trigger フィールドで分岐する。

trigger	起動元	処理
（なし）	EventBridge reminder-schedule	定時の自動催促
poll	EventBridge booking-poller	予約分の実行
broadcast	総務 webhook からの非同期 Invoke	一斉送信
notify	総務 webhook からの非同期 Invoke	提出物削除時の再提出依頼（単発 push）

webhook から reminder を呼ぶのは `InvocationType=Event`（非同期）。LINE の 30 秒制限内に ack を返すため、送信処理の完了を待たない（BS_INF-05 §5）。

5. 既知の課題

#	深刻度	区分	内容	推奨対応	要否
1	中	運用	失敗時の検知手段が無い。催促が送られなくても誰も気付かない（CloudWatch アラーム 0 件）	reminder / reconcile の <code>Errors</code> メトリクスにアラームを設定し SNS 通知（BS_INF-09 §4）	早期対応推奨
2	低	構成	EventBridge → Lambda の DLQ 未設定 。Lambda 側の非同期実行失敗が捨てられる	非同期 Invoke（broadcast / notify）に DLQ を設定すると、送信失敗の取りこぼしを回収できる	将来対応
3	低	運用	祝日・年末年始を考慮しない。25日が休日でも催促が飛ぶ	業務上の実害が小さい（社員は翌営業日に対応すればよい）ため対応しない判断も可	対応不要

6. 費用

EventBridge のルール実行は月100万件まで無料。本システムは 10 分間隔 × 2 ルール = 月約 8,640 回で、無料枠に十分収まる。

7. 関連文書

- 起動される Lambda … BS_INF-02
- `bookings` テーブル … BS_INF-03 DynamoDB設計書 §2
- 監視 … BS_INF-09

BrightStar 統合LINEアシスタント IAM設計書

項目	内容
文書番号	BS_INF-07
文書名	IAM設計書
対象システム	BrightStar 統合LINEアシスタント（5チャンネル）
版数	初版
最終更新	2026-09-19
実機照合	2026-09-19（全ロール・インラインポリシー実査）

1. 方針

項目	設計	理由
ロールの粒度	Lambda 関数ごとに専用ロール （CDK が自動生成）	関数間で権限を共有しない。1 関数の権限追加が他へ波及しない
権限付与	CDK の <code>grant*</code> メソッド（ <code>grantReadWriteData</code> 等）を使用	リソース ARN の手書きミスを防ぎ、リソースとポリシーの対応が CDK 上で自明になる
人間用ロール	作らない （IAM ユーザー <code>sonsik</code> の管理者権限で運用）	運用者 1 名の小規模構成。 複数人運用に移行する場合は見直しが必要 （§5 課題3）
権限境界	設定しない	同上

重要な前提：本システムのアクセス制御は **2 層**ある。 - **アプリ層の認可**=社員名簿の役割（`role / roles`）による機能ゲート。**個人情報の閲覧・ダウンロードはここで制御されている**。 - **インフラ層の IAM**=Lambda がどの AWS リソースに触れるか。

個人情報保護の実効的な統制はアプリ層にあり、そこは役割ベースで正しく実装されている（§3）。IAM 層は `grant*` の性質上やや粗い（§4）。

2. ロール一覧（2026-09-19 実測）

全ロールに共通して `AWSLambdaBasicExecutionRole`（CloudWatch Logs への書き込み）を付与。以下はインラインポリシーの内容。

関数	DynamoDB	S3	SSM	KMS	その他
brightstar-hr-dev-webhook	読み書き12アクション	読み書き	GetParameter	Decrypt	—
brightstar-hr-dev-reconcile	読み書き12アクション	—	GetParameter	Decrypt	—
brightstar-soumu-dev-webhook	読み書き12アクション	読み書き	GetParameter	Decrypt	lambda:InvokeFunction
brightstar-soumu-dev-reminder	読み書き12アクション	—	GetParameter	Decrypt	—
brightstar-shain-dev-webhook	読み書き12アクション	明示6アクション	GetParameter	Decrypt	bedrock:InvokeModel
brightstar-kenshu-dev-LineWebhook	読み書き12アクション	—	GetParameter	Decrypt	bedrock:InvokeModel / secretsmanager:GetSecretValue
EkiCommute-dev-LineWebhookFn	読み書き12アクション	読み書き	GetParameter	Decrypt	bedrock:InvokeModel / geo-places:* / lambda:InvokeFunction

「読み書き12アクション」の内訳（CDK grantReadWriteData が展開するもの）： BatchGetItem / BatchWriteItem / ConditionCheckItem / DeleteItem / DescribeTable / GetItem / GetRecords / GetShardIterator / PutItem / Query / Scan / UpdateItem

個別権限の根拠

権限	付与先	用途
ssm:GetParameter + kms:Decrypt	全関数	LINE チャネルシークレット／トークン（SecureString）の取得。SecureString は KMS 復号を伴うため両方が必要
lambda:InvokeFunction	総務 webhook / 営業	重い処理を別 Lambda へ非同期委譲するため（BS_INF-05 §5）
bedrock:InvokeModel	社員・研修・営業	研修の RAG Q&A、営業の住所正規化。現状 BEDROCK_ENABLED=false で未使用だが権限は付いている
secretsmanager:GetSecretValue	研修	Zoom API 認証情報
geo-places:Geocode / SearchText / SearchNearby	営業	住所 → 最寄駅の変換

3. アプリ層の認可（個人情報保護の実効統制）

本システムが扱う個人情報：氏名・社員番号・所属部署・国籍等の属性・LINE ユーザーID・勤務実績・通勤経路・住所。マイナンバー・銀行口座・在留カードは扱わない。

操作	必要な役割	実装
自分の提出・自分の履歴閲覧	employee（全社員）	本人の userId に紐づくデータのみ参照
全社員の提出状況・未提出一覧・CSV 出力	hr または soumu	総務チャネルの認証ゲート
提出物の一括DL・個別DL・削除	hr または soumu	同上
社員名簿の追加・変更・削除	hr	人事チャネルの認証ゲート
一斉送信	admin 属性を持つ者のみ	役割に加えた追加チェック
研修のコース作成・公開	teacher	研修チャネルの認証ゲート
要員名簿・通勤コスト比較	sales	営業チャネルの認証ゲート

評価：個人情報を横断的に閲覧・取得できる操作（一覧・CSV・一括DL・削除）はすべて総務／人事の役割に限定されており、一般社員・営業・講師は自分の情報以外に到達できない。「ログイン済みなら全ロール可」にはなっていない。

認証の失効：認証行は TTL により当日限りで失効する。翌日は必ず名簿照合が再実行されるため、名簿から削除された退職者は翌日には利用不能になる。加えて日次リコンサイル（BS_INF-06）が到達不可のアカウントの紐付けを解除する。

4. IAM 層の権限評価

観点	評価
関数ごとに専用ロール	✅ できている
不要なサービスへの権限が無い	✅ 概ねできている（各関数が実際に使うサービスのみ）
アクションの絞り込み	⚠️ 粗い 。 <code>grantReadWriteData</code> により、読むだけの関数にも <code>DeleteItem</code> ・ <code>Scan</code> が付く
リソースの絞り込み	⚠️ 粗い 。参照する全テーブルに一律で同じ12アクションが付く

具体例：営業チャネルの Lambda は社員名簿（roster）を「役割を確認するために読む」だけだが、実際には **roster に対する `DeleteItem` 権限**を持っている。人事チャネルの reconcile も同様。

これは CDK の `grantReadWriteData` を使った結果であり、意図的に広げたものではない。**アプリ層で役割ゲートが効いているため実際の業務操作としては到達不能**だが、コードの不具合や将来の改修で誤って削除処理を書いた場合、IAM が最後の防波堤にならない。

社員チャネルのみ S3 権限を明示列挙（`GetObject` / `PutObject` / `DeleteObject` / `ListBucket` / `PutObjectTagging` / `GetObjectTagging`）しており、**この書き方が本来あるべき形**である。

5. 指摘事項

【深刻度：中】【区分：認可】
内容：読み取りしか行わない Lambda にも DynamoDB の `DeleteItem` / `BatchWriteItem` 権限が付いている。
特に営業・研修の Lambda が社員名簿（roster）を削除できる状態。
根拠：最小権限の原則から逸脱。アプリ層の役割ゲートを回避する不具合（例：誤ったハンドラ登録、想定外の入力によるコードパス）が入った場合、IAM が防御層として機能しない。
roster は全チャネルの認証基盤であり、破壊されると全社で bot が使用不能になる。
推奨対応：参照のみの関数は `grantReadData` に変更する。読み書きが必要な関数も、社員チャネルの S3 権限と同様にアクションを明示列挙する。
ブロックカー要否：将来対応（アプリ層の統制が効いているため即時のリスクは低い）

【深刻度：低】【区分：構成】
内容：`bedrock:InvokeModel` が社員・研修・営業に付与されているが、`BEDROCK_ENABLED=false` で機能は未使用。
根拠：使っていない権限は攻撃面を広げる。またリソースが `"*"` 指定。
推奨対応：機能を有効化するまで権限を外す。有効化時は `foundation-model` と `inference-profile` の ARN を明示する（東京リージョンの Claude は `inference-profile` 必須）。
ブロックカー要否：将来対応

【深刻度：中】【区分：構成】
内容：人間（運用者）用の IAM ロールが定義されておらず、管理者権限の IAM ユーザーで直接運用している。
根拠：運用者が 1 名のうちは実害が小さいが、複数人運用・委託先参加の段階で「誰が何をしたか」の追跡と権限分離ができない。個人情報保持するシステムとして、アクセス権者の限定は個人情報保護法上の安全管理措置に該当する。
推奨対応：運用者ロール（読み取り中心）とデプロイロール（CDK 実行）を分離し、IAM ユーザーは `AssumeRole` 経由に切り替える。
ブロックカー要否：複数人運用へ移行する前に必須 / 現状（1名運用）では将来対応

【深刻度：低】【区分：監査】
内容：CloudTrail のデータイベント（S3 オブジェクトアクセス・DynamoDB 項目アクセス）が未設定。
根拠：提出物（個人情報）に誰がいつアクセスしたかの記録が残らない。
管理イベントは既定で記録されるが、オブジェクト単位のアクセスは対象外。
推奨対応：S3 バケット `hr/` プレフィックスのデータイベントを CloudTrail で記録する。
費用はイベント数課金のため、本規模では軽微。
ブロックカー要否：将来対応

6. 費用

IAM 自体は無料。CloudTrail データイベントを有効化する場合のみ、記録イベント数に応じた課金が発生する。

7. 関連文書

- 認証情報の保管と暗号鍵 … BS_INF-08
- 監査ログ … BS_INF-09
- 役割ゲートの機能仕様 … 11 機能仕様書 § 1

BrightStar 統合LINEアシスタント SSM Parameter Store・暗号化設計書

項目	内容
文書番号	BS_INF-08
文書名	SSM Parameter Store・暗号化設計書
対象システム	BrightStar 統合LINEアシスタント（5チャンネル）
版数	初版
最終更新	2026-09-19
実機照合	2026-09-19

1. 機密情報の保管方針

項目	設計	理由
保管先	SSM Parameter Store (SecureString)	Secrets Manager はローテーション機能を持つがパラメータ毎に月額固定費が発生する。LINE のチャンネルシークレット／トークンは自動ローテーション非対応（LINE 側の仕様）で、Secrets Manager の主機能を活かせない。Parameter Store の SecureString は無料で、必要な機能（KMS 暗号化・IAM 制御）は満たす
値の投入	デプロイとは別に手動で 1 回投入。 CDK はパラメータ名のみを参照	CDK のコードにもコンテキストにも値を残さない。cdk.context.json や CloudFormation テンプレートに秘密が焼き込まれる事故を構造的に防ぐ
ソース管理	値はリポジトリ・設計書・記憶ファイルのいずれにも書かない	.gitignore で *.env を除外。設計書にはパラメータ名のみ記載する
例外	Zoom の認証情報のみ Secrets Manager	研修チャンネルの既存実装を踏襲

2. パラメーター一覧（2026-09-19 実測・名前のみ）

すべて SecureString。

パラメータ名	用途
/brightstar-kenshu/dev/line/secret	研修チャンネルの署名検証用シークレット
/brightstar-kenshu/dev/line/token	研修チャンネルのアクセストークン
/brightstar-hr/dev/line/secret	人事チャンネル
/brightstar-hr/dev/line/token	人事チャンネル
/brightstar-soumu/dev/line/secret	総務チャンネル
/brightstar-soumu/dev/line/token	総務チャンネル
/brightstar-shain/dev/line/secret	社員チャンネル
/brightstar-shain/dev/line/token	社員チャンネル
/eki-commute/dev/line/channel-secret	営業チャンネル
/eki-commute/dev/line/channel-access-token	営業チャンネル

命名は `/{{app}}/{{stage}}/line/{secret|token}`（営業のみ既存実装の都合で `channel-` 接頭辞）。

シークレットの二重用途

LINE チャンネルシークレットは、署名検証に加えてダウンロード用短縮 URL の HMAC 署名鍵にも流用している（BS_INF-04 § 4、BS_INF-05 § 4）。

【深刻度：低】【区分：鍵管理】

内容：1 つの鍵を「LINE 署名検証」と「自前の URL 署名」の 2 用途で共用している。
根拠：LINE 側の都合でチャンネルシークレットを再発行すると、発行済みのダウンロードリンクが一斉に無効になる。逆に URL 署名方式を変えたいときに LINE 側へ影響が出る。
鍵の用途分離（1鍵1用途）の原則から外れる。
推奨対応：URL 署名用に専用のパラメータ（例 `/brightstar-soumu/dev/dl/signing-key`）を追加する。

既存リンクは exports/ が 3 日で消えるため、移行の影響は軽微。
ブロッカー要否: 将来対応

3. 暗号鍵（KMS）方針

AWS 管理キー（サービス既定の暗号化）で運用し、CMK（顧客管理キー）は作らない。

理由：CMK の価値はキーポリシーによるアクセス分離・独自のローテーション周期・クロスアカウント共有だが、本システムは単一アカウント・単一ステージ・運用者1名でいずれにも該当しない。CMK はキー毎の固定費と、鍵の無効化事故＝データ復号不能というリスクを伴う。要件が出た時点で導入する。

サービス別の暗号化状況（2026-09-19 実測）

サービス	暗号化方式	キー	出典
SSM Parameter Store	SecureString	aws/ssm （AWS 管理キー）	本書 § 2
DynamoDB（brightstar 系13表）	保存時暗号化	AWS 管理キー	BS_INF-03 § 2
DynamoDB（営業2表）	保存時暗号化	AWS 所有キー（監査性が劣る）	BS_INF-03 § 2
S3（brightstar-hr-dev-…）	SSE-S3（AES-256）	S3 管理	BS_INF-04 § 2
Secrets Manager（Zoom）	既定暗号化	aws/secretsmanager	研修チャンネル
CloudWatch Logs	既定暗号化	サービス管理	BS_INF-09
転送中	TLS（HTTPS）のみ	—	LINE API・AWS SDK とも HTTPS

CMK へ移行する条件（いずれかが発生したら本書改訂）

- 監査（ISO 27001 等）で鍵管理・ローテーション周期の明示を求められた場合。
- 複数アカウント構成へ移行し、アカウント間でデータ共有が必要になった場合。
- マイナンバー・銀行口座など、より機微な情報を扱う範囲へ拡張する場合。

現状の取扱情報は氏名・所属・勤務実績・通勤経路までで、特定個人情報は扱っていない。

4. 既知の課題

〔深刻度：高〕〔区分：機微情報 / 構成〕

内容：研修チャンネルの企業微信（WeCom）認証情報が Lambda の環境変数に平文で保持されている。

対象：WECOM_SECRET / WECOM_AES_KEY / WECOM_TOKEN / WECOM_RELAY_AUTH。

根拠：Lambda の GetFunctionConfiguration 権限を持つ者・CloudFormation テンプレートの閲覧者が値を平文で読める。SSM SecureString に統一するという本システムの方針からも逸脱している。

また CDK デプロイ時に -c で渡すため、シェル履歴・CI ログに残る経路がある。

推奨対応：SSM SecureString（/brightstar-kenshu/dev/wecom/*）へ移し、Lambda は起動時に取得する（LINE の secret/token と同じパターンが既に実装済みなので流用できる）。

移行後は企業微信側で認証情報を再発行し、平文で流通した旧値を無効化する。

ブロッカー要否：企業微信連携を継続するなら早期対応。LINE 側の運用には影響しない。

〔深刻度：中〕〔区分：鍵管理 / 運用〕

内容：研修スタックの再デプロイ時、上記の値を -c で渡し忘れると環境変数が空で上書きされ、企業微信連携が無言で停止する。

根拠：CDK は context 未指定時に空文字を既定値としているため、エラーにならず機能だけが止まる。

推奨対応：SSM 化（上記）により構造的に解消する。それまでは、稼働中 Lambda の環境変数から

値を読み戻して -c を組み立てる手順を必ず使う（BS_INF-10 § 4）。

ブロッカー要否：手順で回避中（運用ルールとして明文化済み）

5. 費用

- SSM Parameter Store（Standard + SecureString）：無料
- AWS 管理キーによる暗号化：無料（KMS API 呼び出しは Lambda 起動時のパラメータ取得のみで、無料枠内）
- Secrets Manager（Zoom 用 1 件）：月額約 \$0.4

CMK を採用した場合はキー毎に月額 \$1 + API 従量が追加される。

6. 関連文書

- 権限（誰がパラメータを読めるか）… BS_INF-07 IAM設計書
- 署名付き URL の仕組み… BS_INF-04 S3設計書 § 4
- デプロイ時の context 指定… BS_INF-10 § 3

BrightStar 統合LINEアシスタント 監視・ログ設計書

項目	内容
文書番号	BS_INF-09
文書名	監視・ログ設計書
対象システム	BrightStar 統合LINEアシスタント（5チャンネル）
版数	初版
最終更新	2026-09-19
実機照合	2026-09-19

1. 方針

項目	設計	理由
ログ出力	Lambda の標準出力 → CloudWatch Logs	追加のエージェント・転送基盤を持たない
保持期間	30日（営業チャンネルのみ 14日）	ログに氏名・LINE ユーザーID が含まれる（§ 3）ため無期限保持にしない。一方、業務が月次サイクル（25日・28日の催促）であるため、直近1サイクルを追える長さが必要。この2条件の交点が 30 日
集約	各 Lambda のロググループのみ。集約基盤は持たない	関数が 11 本と少なく、CloudWatch Logs Insights で横断検索できる
メトリクス監視	現状ほぼ未実装（§ 4）	—

2. ロググループと保持期間（2026-09-19 実測）

ロググループ	保持期間
/aws/lambda/brightstar-hr-dev-webhook	30日
/aws/lambda/brightstar-hr-dev-reconcile	30日
/aws/lambda/brightstar-soumu-dev-webhook	30日
/aws/lambda/brightstar-soumu-dev-reminder	30日
/aws/lambda/brightstar-shain-dev-webhook	30日
/aws/lambda/brightstar-kenshu-dev-*（4本）	30日
/aws/lambda/EkiCommute-dev-*（2本）	14日
CDK 補助関数（LogRetention / CustomResource 等）	1日 または 30日

無期限保持のロググループは 0 件（19 件すべてに保持期間を設定済み）。

実装（CDK）

```
// TypeScript CDK（人事・総務）
new lambda.Function(this, "WebhookFunction", {
  ...
  // ログに氏名等が出るため無期限保持にしない（月次業務1周期を追える30日）
  logRetention: logs.RetentionDays.ONE_MONTH,
});

# Python CDK（研修・社員）
lambda_.Function(
  ....
```

```
log_retention=logs.RetentionDays.ONE_MONTH,
)
```

落とし穴： `logRetention` は CDK v2 で **deprecated** (v3 で削除予定)。非推奨でない書き方は `logGroup` に明示的な `LogGroup` を渡す方式だが、**既存のロググループが存在する状態でこれを使うと「already exists」でデプロイが失敗する**。移行するにはロググループを一旦削除する＝既存ログを失う。本システムは**既存ログを保持したかったため `logRetention` を選択した**。CDK v3 移行時に、ログを捨ててよいタイミングで `logGroup` 方式へ切り替える。

2026-09-19 に実施した是正

内容	件数
無期限保持 → 30日へ変更 (CDK に <code>logRetention</code> 追加・4スタック再デプロイ)	9 ロググループ
CDK 補助関数のロググループに 30日を設定 (CLI)	3 ロググループ
旧スタック由来の孤立ロググループを削除	5 ロググループ (計 791KB・最終ログ 2026-06)

削除したのも： `brightstar-dev-line-webhook` / `brightstar-dev-reminder` / `brightstar-dev-web` / `brightstar-dev-webhook` (削除済みスタックの残骸)、`brightstar-hr-dev-reminder` (機能が総務チャネルへ移管され関数が消滅)。いずれも**対応する Lambda 関数が存在しないことを確認**のうえ削除した。

3. ログに出力される情報

出力箇所	内容	個人情報
<code>[UNFOLLOW] {userId} name={氏名}</code>	bot のブロック／削除検知	氏名・LINE ユーザーID
<code>[RECONCILE] cleared empId={社員番号} name={氏名}</code>	日次リコンサイルでの紐付け解除	氏名・社員番号
<code>push fail: {userId} {エラー}</code>	LINE push の失敗	LINE ユーザーID
<code>reminder sent=N period=... type=...</code>	催促の送信件数	なし (統計のみ)
<code>route error: {例外}</code>	ハンドラ内例外	例外内容による

[深刻度：中] [区分：個人情報]

内容：CloudWatch Logs に氏名・社員番号・LINE ユーザーID が平文で出力されている。

根拠：個人情報保護法の安全管理措置上、個人データは利用目的の範囲で必要最小限に扱う必要がある。

障害調査目的でログに氏名を残す必然性は低く (社員番号または `userId` の下4桁で追跡可能)、ログ閲覧権限を持つ者すべてが氏名を閲覧できる状態になる。

推奨対応：ログ出力を識別子のみに変更する (`name=` を落とす、`userId` は末尾数桁にマスクする)。

保持期間 30 日の設定により露出期間は限定済みだが、出力自体を減らすのが本筋。

ブロッカー要否：将来対応 (保持期間の設定により当面のリスクは低減済み)

4. メトリクス監視

現状：CloudWatch アラーム 0 件

[深刻度：高] [区分：運用]

内容：アラームが 1 つも設定されておらず、障害を検知する手段が無い。

根拠：本システムの中核機能である「月末の自動催促」は月2回しか動かない。

`reminder` Lambda が失敗しても誰にも通知されず、「催促が来ないまま締切を過ぎた」ことに人が気付くまで分からない。

同様に、`webhook` が全件エラーになっても LINE 利用者が個別に報告するまで検知できない。

推奨対応：下表の最小構成のアラームを設定し、SNS で管理者へ通知する。

ブロッカー要否：早期対応 (本番相当で運用しているため)

推奨する最小構成

#	対象	メトリクス	しきい値	意味
1	<code>brightstar-soumu-dev-reminder</code>	<code>Errors</code>	1回以上 / 5分	催促が送れていない＝業務が止まる。最優先
2	<code>brightstar-hr-dev-reconcile</code>	<code>Errors</code>	1回以上 / 日次	名簿の到達性点検が失敗
3	全 <code>webhook</code> (3本)	<code>Errors</code>	5回以上 / 5分	恒常的なエラー (単発は LINE 側の一時障害もあるため閾値を持たせる)
4	全 <code>webhook</code>	<code>Throttles</code>	1回以上	同時実行数の上限に当たっている
5	<code>brightstar-soumu-dev-reminder</code>	<code>Invocations</code>	25日・26日に 0 件	定時催促が「起動すらしていない」ことの検知 (EventBridge 側の異常)

5 は「動かなかったこと」を検知する設計。Errors だけでは、EventBridge ルールが無効化された場合や Lambda が呼ばれなかった場合を捕まえられない。

通知先

SNS トピック 1 本を作り、管理者のメールアドレスを購読させるのが最小構成。LINE への通知（総務チャンネルへ push）も技術的には可能だが、LINE 自体の障害時に届かないため、監視通知は LINE と独立した経路にすべき。

5. 費用

項目	現状
CloudWatch Logs 取り込み	月あたり数十MB 程度（最大のログは研修 reminder の 7MB／30日）
CloudWatch Logs 保管	30日保持により上限が定まった。無期限だった従来は累積し続ける構造だった
アラーム	未設定（設定しても 10 個までは無料枠）

保持期間の設定は、個人情報の観点だけでなく保管コストの上限を確定させる効果もある。

6. 既知の課題（まとめ）

#	深刻度	内容	要否
1	高	CloudWatch アラームが 0 件。催促失敗・webhook 障害を検知できない	早期対応
2	中	ログに氏名・社員番号が平文で出力される	将来対応
3	中	CloudTrail のデータイベント未設定。提出物へのアクセス監査ができない（BS_INF-07 §5）	将来対応
4	低	logRetention が CDK deprecated API。v3 移行時に書き換えが必要	将来対応
5	低	営業のみ 14日で他と不揃い	対応不要（業務性質が異なる）

7. 関連文書

- Lambda 一覧 … BS_INF-02
- 定期実行と失敗検知 … BS_INF-06 EventBridge設計書 § 5
- 監査ログ … BS_INF-07 IAM設計書 § 5

BrightStar 統合LINEアシスタント CDK構成管理・デプロイ設計書

項目	内容
文書番号	BS_INF-10
文書名	CDK構成管理・デプロイ設計書
対象システム	BrightStar 統合LINEアシスタント（5チャンネル）
版数	初版
最終更新	2026-09-19
実機照合	2026-09-19

1. 方針

項目	設計	理由
IaC	AWS CDK v2 (aws-cdk-lib ^2.150.0)	手動作成リソースを作らない。全構成をコードで再現可能にする
スタック分割	チャンネルごとに独立したスタック・独立した CDK アプリ	1 チャンネルの改修が他チャンネルのデプロイを要求しない。障害の波及も防ぐ
言語	TypeScript (人事・総務・営業) / Python (研修・社員)	各チャンネルが別時期に別プロジェクトとして作られた経緯。統一していない (§ 7 課題1)
コード同梱	Code.fromAsset でリポジトリのディレクトリをそのまま zip 化	ビルド工程を挟まないため、 リポジトリ = デプロイ内容 が常に成立する (§ 5)
機密の注入	SSM SecureString のパラメータ名のみ CDK に持たせる	値を CDK・テンプレート・context に置かない (BS_INF-08 § 1)
bootstrap	ap-northeast-1 に実施済み (CDKToolkit)	再実行不要

2. スタック構成

スタック名	チャンネル	言語	ディレクトリ	作成する主なリソース
brightstar-kenshu-dev	研修	Python	kenshu/cdk	Lambda×4 / API GW / DynamoDB×6 / EventBridge×1
BrightstarHr-dev	人事	TypeScript	jinja/cdk	Lambda×2 / Function URL / DynamoDB×4 (共有) / S3 (共有) / EventBridge×1
EkiCommute-dev	営業	TypeScript	eigyo/infra	Lambda×2 / Function URL×2 / DynamoDB×2 / Layer
BrightstarSoumu-dev	総務	TypeScript	soumu/cdk	Lambda×2 / Function URL / DynamoDB×2 / EventBridge×2
brightstar-shain-dev	社員	Python	shain/cdk	Lambda×1 / Function URL / DynamoDB×1

依存関係

BrightstarHr-dev (人事)

└─ roster / auth / employees / submissions / S3バケット を作成

↑ 他の 4 スタックが「テーブル名の文字列」で参照 (Export/Import は使わない)

項目	内容
初回デプロイ順序	人事を最初にデプロイする (共有テーブル・バケットを作るため)。以降は任意
更新時の順序	制約なし。各スタックを独立に更新できる
削除時の注意	人事スタックを削除すると他4チャンネルが全滅する。全テーブルが RemovalPolicy.DESTROY のため、データも消える (BS_INF-03 § 7 課題2)

Export/Import を使わない理由：スタック間に削除順序の依存が生まれ、人事スタックを更新するたびに参照側がロックされるため。名前参照なら疎結合を保てる。

3. デプロイコマンドと必須 context

```
# 研修 (Python CDK) - WeCom 系の context 必須 ( § 4 )
cd kenshu/cdk && npx cdk deploy brightstar-kenshu-dev --require-approval never -c weComCorpId=... (18項目)

# 人事 (TypeScript CDK)
cd jinja/cdk && npm install && npx cdk deploy BrightstarHr-dev --require-approval never ¥
-c masterHrPrefix=sonsik -c mailProofreadUrl="https://sons02-relay.tail5a0084.ts.net:8443/"

# 総務
cd soumu/cdk && npm install && npx cdk deploy BrightstarSoumu-dev --require-approval never ¥
-c masterHrPrefix=sonsik

# 社員 (context 不要)
cd shain/cdk && npx cdk deploy brightstar-shain-dev --require-approval never

# 営業 (Layer をビルドしてから)
cd eigyo/infra && npm install && python build.py && npx cdk deploy EkiCommute-dev --require-approval never
```

⚠ context を渡し忘れるとどうなるか

CDK は tryGetContext の戻りが undefined のとき既定値 (多くは空文字) を使う。エラーにならず、環境変数が空で上書きされる。

スタック	必須 context	渡し忘れた場合
研修	<code>weComCorpId</code> / <code>weComToken</code> / <code>weComAesKey</code> / <code>weComAgentId</code> / <code>weComSecret</code> / <code>weComRelayUrl</code> / <code>weComRelayAuth</code> / <code>weComKfOpenKfid</code> / <code>teacherOpenids</code> / <code>teacherSignupCode</code> ほか計18	企業微信連携が無言で停止。講師の signup code も失われる
人事	<code>masterHrPrefix</code> / <code>mailProofreadUrl</code>	メール校正ツールへのリンクが消える
総務	<code>masterHrPrefix</code>	マスター権限判定が効かなくなる
社員・営業	なし	—

4. 研修スタックの安全な再デプロイ手順

研修の設定値は **Lambda 環境変数**にしか存在しない（SSM 化されていない — BS_INF-08 §4）。そのため、稼働中の **Lambda** から値を読み戻して **context** を組み立てる。

```
# 1. 現在の環境変数を取得
$FN = "brightstar-kenshu-dev-LineWebhookFunctionE63D95CF-1qfCxM64IodC"
$before = aws lambda get-function-configuration --function-name $FN --region ap-northeast-1 `
    --query "Environment.Variables" --output json | ConvertFrom-Json

# 2. ctxキー → env変数名 の対応表から -c 引数を生成（値をスクリプトに書かない）
#    weComCorpId→WECOM_CORP_ID, weComToken→WECOM_TOKEN, ... 計18項目

# 3. cdk deploy @ctxArgs

# 4. デプロイ後に before/after を突合し、失われた変数が無いことを確認
```

手順の要点：値をスクリプト・シェル履歴に書かず、AWS → CLI の経路だけで流す。デプロイ後の**突合確認を必ず行う**（2026-09-19 実施時は 34 変数すべて保持を確認）。

この手順は SSM 化（BS_INF-08 §4）により不要になる。恒久対策はそちら。

5. ソースとデプロイの同期検証

`Code.fromAsset` の性質上リポジトリ＝デプロイ内容だが、**納品時に機械的に検証**する。

手順

- 1. 各 Lambda の `Code.Location`（S3 署名付き URL）から配布 zip を取得・展開
- 2. リポジトリ側の対応ソースと **ファイル単位の md5 比較**
- 3. 営業のみ `build.py` のコピー規則で「zip 内パス → git 管理下の元ファイル」へ読み替えて突合（`eigyo/infra/build/` は `.gitignore` のため、build 成果物と比較しても意味がない）

2026-09-19 実施結果：全11関数・122ファイルが一致（差分なし）

チャネル	関数数	ファイル数	結果
研修	4	24	一致
人事	2	15	一致
総務	2	15	一致
社員	1	43	一致
営業	2	25	一致

リポジトリ（`main`）は同時点で未コミット変更・未プッシュコミットともに無し。

6. 落とし穴集

#	内容	対策
1	研修の再デプロイで企業微信設定が消える	§4 の手順を必ず使う
2	Function URL は削除・再作成で URL が変わる	変わったら LINE Developers 側の Webhook URL を手で更新する。通常のスタック更新では変わらない

#	内容	対策
3	<code>logRetention</code> が CDK v2 で deprecated	既存ログを保持するため意図的に使用。v3 移行時に <code>logGroup</code> 方式へ (BS_INF-09 §2)
4	<code>git-bash</code> から / 始まりの引数を渡すと Windows パスに書き換えられる	SSM パラメータ名 (<code>/brightstar-hr/...</code>) ・ロググループ名 (<code>/aws/lambda/...</code>) を扱う AWS CLI は PowerShell で実行する
5	Windows コンソールが GBK で CJK 出力が壊れる	<code>PYTHONUTF8=1</code> を付ける、または <code>chcp 65001</code>
6	vendorizing 先の直し忘れ	社員チャネルの提出処理は <code>shain/lambda/jinji/common/</code> が実体。 <code>jinji/lambda/common/</code> だけ直しても反映されない
7	営業は <code>python build.py</code> を先に実行しないと古いコードがデプロイされる	デプロイ手順に含める (§3)

7. 既知の課題

#	深刻度	内容	推奨対応	要否
1	低	CDK の言語が TypeScript / Python で混在	実害は小さい (各スタックが独立しているため)。統一は書き直しコストに見合わない	対応不要
2	中	CI/CD が無い 。デプロイは手元の PC から手動実行	手順の属人化・デプロイ漏れのリスク。GitHub Actions + OIDC で <code>cdk deploy</code> を自動化できる	将来対応
3	中	<code>cdk diff</code> を必ず確認する運用が明文化されていない	デプロイ前の <code>cdk diff</code> を手順に組み込む	早期対応 (運用ルール)
4	低	ステージが <code>dev</code> のみ。本番相当を <code>dev</code> で運用している	環境分離が必要になった時点で <code>-c stage=prod</code> で並行構築できる設計にはなっている	将来対応

8. 関連文書

- 全設計書の索引 … [README](#)
- リソース実体 … BS_INF-02 / BS_INF-03 / BS_INF-04
- 機密の注入 … BS_INF-08